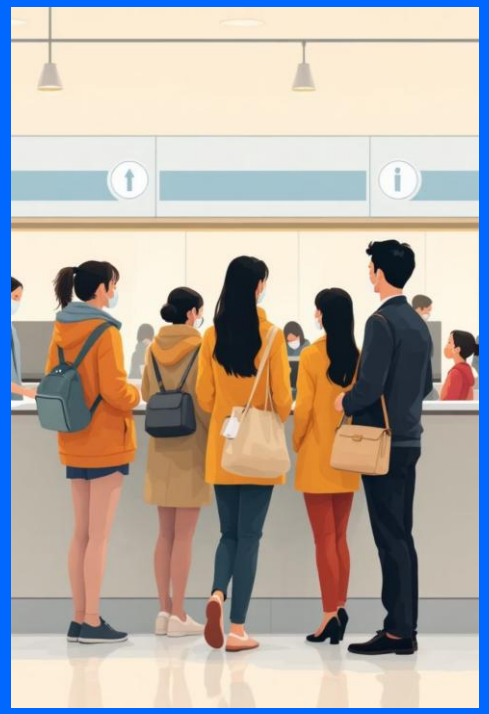
Estrutura de Dados não Lineares

UNIVERSIDADE DO VALE DO SAPUCAÍ – UNIVAS – POUSO ALEGRE/MG
PROFESSOR: RAFFAEL CARVALHO

Fila Dinâmica em Java

Primeiro a Chegar, Primeiro a Ser Atendido



O Conceito FIFO

First In, First Out

A Fila segue a regra de que o primeiro elemento a entrar é o primeiro a sair, exatamente como uma fila no caixa eletrônico.

Se a Pilha é o botão "Desfazer", a Fila é a ordem natural de atendimento: quem chega chega primeiro, é atendido primeiro.

Ponteiros Múltiplos para Eficiência

Início (Head)

Ponteiro que aponta para o primeiro elemento da fila, onde as onde as remoções acontecem (Dequeue)

Fim (Tail)

Ponteiro que aponta para o último elemento da fila, onde as onde as inserções acontecem (Enqueue)

Gerenciar dois ponteiros permite que ambas as operações rodem em tempo constante O(1), evitando percorrer toda a estrutura.

A Classe NoFila

O nó base que vive no Heap, armazenando dados e referências

```
public class NoFila {
    public int dado;
    public NoFila proximo;

    public NoFila(int dado) {
        this.dado = dado;
        this.proximo = null;
    }
}
```

Cada nó contém um valor inteiro e uma referência para o próximo nó na sequência.

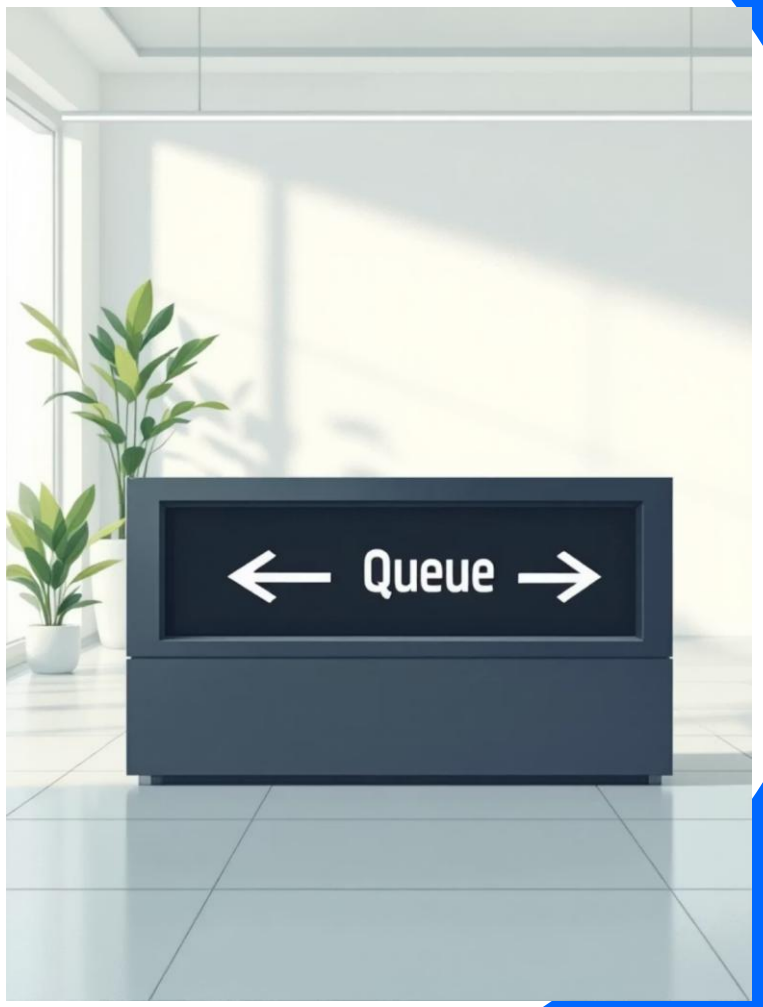
Construtor e Verificação

Estrutura Inicial

```
public FilaDinamica() {
    this.inicio = null;
    this.fim = null;
}

public boolean isEmpty() {
    return inicio == null;
}
```

A fila começa vazia com ambos ponteiros nulos. O método isEmpty() verifica se não há elementos. há elementos.



Operação Enfileirar (Enqueue)

Inserir no Final da Fila

1

Fila Vazia

Novo nó se torna simultaneamente início e fim

2

Fila com Elementos

Fim atual aponta para novo nó, que se torna o novo fim

```
public void enfileirar(int valor) {
 NoFila novoNo = new NoFila(valor);
 if (isEmpty()) {
 inicio = novoNo;
 fim = novoNo;
 } else {
 fim.proximo = novoNo;
 fim = novoNo;
 }
}
```

Operação Desenfileirar (Dequeue)

Remover do Início da Fila

Processo

- Guarda o valor do início
- Move início para o próximo nó
- Se esvaziar, zera o ponteiro fim

Complexidade

O(1) - Tempo constante, pois não precisa percorrer a estrutura.

Operação observar (peek)

Observar Sem Remover

A operação observar() permite visualizar quem é o próximo a ser atendido sem efetivamente efetivamente remover o elemento.

```
public int obsevar() {
    if (isEmpty()) {
        throw new RuntimeException("Fila vazia!");
    }
    return inicio.dado;
}
```

Essencial para verificar o próximo elemento antes de processá-lo.

Teste Prático

01

Inserir 10, 20, 30

Fila: [10 → 20 → 30]

02

Peek: 10

Próximo a ser atendido

03

Dequeue: 10

Remove o primeiro

04

Dequeue: 20

Remove o próximo

05

Inserir 40

Fila: [30 → 40]

06

Dequeue tudo

Fila esvazia completamente

Fila vs. Pilha: Irmãs Separadas Separadas

Fila (FIFO)

Primeiro a entrar, primeiro a sair. Inserção no fim, remoção do início.

Pilha (LIFO)

Último a entrar, primeiro a sair. Inserção e remoção do topo.

Ambas são implementações irmãs separadas apenas pela regra de negócio sobre negócio sobre onde o dado entra e por onde ele sai.

Desafio Prático 1: Simulador de Fila de Impressão (Spooler)

Em ambientes corporativos e laboratórios de faculdade, várias pessoas enviam documentos para a mesma impressora ao mesmo tempo. O sistema operacional usa uma operacional usa uma estrutura FIFO (Fila) para garantir que o primeiro arquivo enviado seja o primeiro a ser impresso.

A Missão:

01

Importe as classes `NoFila` e `FilaDinamica` que construímos na aula.

02

Crie uma nova classe chamada `GerenciadorImpressao`.

03

Essa classe deve ter como atributo a sua `FilaDinamica` (que guardará o ID numérico dos documentos).

04

Implemente os seguintes métodos:

- `enviarDocumento(int idDoc)`: Deve colocar o documento na fila e uma mensagem de confirmação.
- `processarImpressao()`: Deve retirar o próximo documento da fila e simular simular a sua impressão no console. Lembre-se de verificar se a fila está vazia! está vazia!
- `exibirFilaPendente()`: Mostra todos os documentos que ainda aguardando.

Desafio Prático 2: Sistema de Atendimento de Help Desk

Em uma empresa de tecnologia, quando os funcionários têm problemas nos computadores, eles abrem um "Chamado" (Ticket) no sistema de suporte. A equipe de TI tem suporte. A equipe de TI tem a regra estrita de atender os chamados na ordem de chegada (FIFO - First In, First Out), para ser justa com quem está esperando há mais esperando há mais tempo.

A Missão:

01

Importe as classes `NoFila` e `FilaDinamica` que criamos.

02

Crie uma nova classe chamada `GerenciadorAtendimento`.

03

Essa classe usará a `FilaDinamica` para gerenciar os números de identificação (IDs) dos chamados.

04

Implemente os seguintes métodos:

- `abrirChamado(int idChamado)`: O usuário reportou um problema. Coloque o ID na fila e exiba uma mensagem.
- `verificarProximoChamado()`: O técnico quer saber qual é o próximo problema que ele vai resolver, mas sem retirá-lo da fila ainda (use o peek()). peek()).
- `atenderChamado()`: O técnico começou a trabalhar. Retire o e mostre qual ID está sendo resolvido.
- `exibirPainel()`: Mostra a fila atualizada no telão do departamento de TI. de TI.

Bibliografia Básica

- CELES, Waldemar. Introdução a Estruturas de Dados - Com Técnicas de Programação em C. Grupo GEN, 2016. E-book. ISBN 9788595156654.
- PUGA, Sandra; RISSET, Gerson. Lógica de programação e estruturas de dados - com aplicações em Java. 2. ed. São Paulo: Pearson, 2006.
- GOODRICH, Michael T.; TAMASSIA, Roberto. Estruturas de dados e algoritmos em Java. Grupo A, 2013. E-book. ISBN 9788582600191.